

# Bottleneck-Guided Keccak Acceleration on an RV32 SIMT GPU

Anonymous authors  
Working draft — September 2026

**Abstract**—We study bottleneck selection and instruction granularity for post-quantum cryptography on an RV32 Vortex SIMT GPU. Software ablation attributes 65.65% of an instrumented ML-KEM-768 round trip to Keccak-f[1600] and 12.67% to forward and inverse number-theoretic transforms, giving elimination bounds of 2.911 and 1.145. We retain C and RV32 assembly baselines, then implement THETA, RHOPI, and CHII/IOTA as six single-destination instructions. Twenty-five lanes hold a state in ordinary registers. A whole-round register instruction pair and a memory-pointer permutation engine provide comparison points. On the same 8-warped, 32-thread processor, stage instructions improve RTL permutation throughput by 12.91 times over shuffle-only SG25. With the final cooperative NTT and arithmetic path fixed, the stage backend improves complete ML-KEM XRT cycles by 5.258 times at one request and 3.646 times at eight requests over serial software; the instruction-only gains over the identical shuffle mapping are 1.462 and 1.552 times. A separate 200 MHz board sweep with matched complete software mappings measures joint Keccak, NTT, and arithmetic speedups of 1.910–2.365 times at one request and 1.914–2.126 times at eight across six ML-KEM/ML-DSA parameter sets. At a common 250 MHz target, the stage extension increases full-core LUTs by 1.22% and flip-flops by 0.73%. A matched RV32IM control gives both Stage and whole round fully expanded round code. Whole round retains 3.300 times the permutation throughput, but improves complete ML-KEM by only 1.028 and 1.033 times. A matched integer-only RV64 port halves the Keccak collective issue count. At application scope it reduces Stage/KROUND instructions by 7.00%/5.73% and one-request cycles by 4.07%/2.98%, while eight-request cycles rise slightly. Collective granularity trades pipeline storage against benefits that shrink substantially at application scope.

**Index Terms**—post-quantum cryptography, ML-KEM, ML-DSA, Keccak, RISC-V, SIMT, instruction-set extension, FPGA

## I. INTRODUCTION

ML-KEM and ML-DSA standardize module-lattice mechanisms for key establishment and digital signatures [1], [2]. Their implementations combine polynomial arithmetic with SHA-3 and SHAKE [3]. A processor designer can accelerate an NTT butterfly, a Keccak stage, or an entire permutation, but the most parallel primitive need not provide the largest application benefit. The baseline software, the distribution of work among lanes, and the fraction of time outside the accelerated primitive all influence the result.

An extensible GPU makes this choice particularly consequential. Vortex supplies an open RISC-V SIMT hardware and software stack [4], allowing custom instructions to be evaluated through software, functional simulation, processor RTL, and synthesis. A pointer instruction can send a memory-

resident Keccak state to a dedicated processing element (PE). Alternatively, a subgroup can hold the state in ordinary lane registers and cooperate through reductions and fixed gathers. Both may be fast, but they exercise different parts of the GPU and impose different integration costs.

This work asks three questions. First, how much of an unaccelerated PQC request is actually attributable to Keccak and NTT, with a stated input and denominator? Second, how much benefit comes from a cooperative software mapping before adding an instruction? Third, how do stage-level and whole-round collectives compare with a pointer PE after accounting for complete requests and full-core implementation cost?

Our contributions are as follows.

- 1) We assemble a reproducible baseline study covering reference C, optimized RV32 Keccak assembly, request batching, cooperative software NTT, and component ablation. The study distinguishes measured cycle differences from extrapolated or algebraic quantities.
- 2) We implement a stage-factored subgroup interface with ordinary GPR state, explicit RV32 low/high destinations, an RV64 full-word port, and a full-warped contract. All eight combinations of software and instruction stages are evaluated on the same hardware.
- 3) We implement a whole-round GPR comparator and evaluate it alongside the stage design and pointer PE. The comparison connects instruction granularity to RTL throughput, ML-KEM latency, memory-operation counts, FPGA full-core area/timing, and isolated ASIC cell area.
- 4) We select a register-resident W32 NTT with a shared 16-multiplier bank, validate its complete-KEM behavior, and hold this implementation fixed in the final Keccak performance and full-core area comparison. A matched RV64 portability experiment measures both complete-KEM behavior and implementation cost without changing the narrow NTT arithmetic. Direct final-configuration intervals separate permutation, sponge, transform, and polynomial costs.

We do not claim the 25-thread state layout or whole-round warped collectives as new concepts. ML-Cube uses 25-thread Keccak with shuffles [5], and an NVIDIA patent application describes a 64-bit warped-collective Keccak round [6]. Our focus is the explicit RV32/RV64 interface, its open implementation, and the controlled comparison of granularities. The measured

results do not select one universal winner: the whole-round unit leads the GPR permutation test, while the pointer PE leads the eight-request ML-KEM test.

## II. BACKGROUND AND EXPERIMENTAL SCOPE

### A. PQC work and two forms of parallelism

The measured ML-KEM-768 request includes key generation, encapsulation, and decapsulation; the ML-DSA-65 round trip includes key generation, signing, and verification. The software is based on mlkem-native and mldsa-native [7], [8]. We call the number of concurrent requests  $M$ . Increasing  $M$  offers independent work to the warp scheduler without shortening a particular request’s dependency chain.

We use  $L$  for software lanes assigned to work within a request. In the historical x4 backend, these lanes process independent hash streams exposed by the library API. In SG25, lanes cooperate on *one* permutation: lane  $t = x + 5y$  holds the 64-bit word  $A[x, y]$ , for  $0 \leq x, y < 5$ . These mappings have different synchronization and storage requirements. Neither their speedups nor their resource counts can be multiplied together without an experiment that combines them.

A Keccak-f[1600] permutation consists of 24 rounds. Each round computes column parity and the THETA correction, rotates and rearranges words in RHOPI, then applies CHI and the IOTA round constant. This produces structured cross-lane communication: five-word column reductions, a fixed permutation, and three-word row gathers. The RV32 interface stores each word in two 32-bit registers, avoiding a change to the core-wide integer width.

### B. Measurement families and denominators

Table I separates the measurement families. Except for the 200 MHz board sweep, cycle results are from simulation. FPGA area and timing numbers come from routed out-of-context `VX_core_top` or isolated-unit implementations targeting the Alveo V80 part. We preserve historical software experiments to explain the design choice, but use the 8-warp, 32-thread experiments for the collective comparison.

The principal collective configuration has a 32-lane ALU and LSU, 16 KiB instruction and data caches, and 16 KiB local memory. L2/L3 are disabled. Historical cache sweeps explicitly vary L2; their numbers are not merged with the main configuration. SimX provides functional and timing models, processor RTL simulation provides cycle comparisons, and XRT simulation checks integration through the accelerator interface.

For an equal-sized batch, speedup is

$$S_{\text{same } M} = \frac{C_{\text{baseline}}(M)}{C_{\text{candidate}}(M)}. \quad (1)$$

For scaling a backend from one request to  $M$ , throughput gain is

$$G_M = \frac{MC(1)}{C(M)}. \quad (2)$$

These are different denominators. Whole-launch cycles include device-side launch setup and teardown; they exclude host

transfers. Phase cycles are intervals around the cryptographic calls. A phase total cannot silently replace a whole-launch denominator.

## III. SOFTWARE BASELINES AND BOTTLENECK ATTRIBUTION

### A. ML-KEM: Keccak, transforms, and the residual

The historical ML-KEM profile uses the same FIPS 203 known-answer-test coins as its corresponding baseline. It executes 144 scalar permutations, including the scalar calls made through x4 fallback, and 15 forward plus nine inverse transforms. The x4 counter records 24 wrapper calls; *it is not added as another  $4 \times 24$  permutations* to the scalar count.

We remove either the permutation body or the forward/inverse transform bodies through native hooks. The cryptographic outputs of these ablated programs are intentionally invalid. They are used only to estimate component cost, with the instrumented baseline as the denominator:

$$f_i = \frac{C_0 - C_{-i}}{C_0}, \quad S_{\infty, i} = \frac{C_0}{C_{-i}} = \frac{1}{1 - f_i}. \quad (3)$$

The removed bodies retain the surrounding invocation path. The cycle differences therefore include consequences for scheduling and memory, rather than being a count of executed ALU operations alone.

Table II gives 21,986,475 removed cycles for Keccak and 4,243,182 for NTT/INTT. Relative to 33,492,153 profile cycles, the shares are 65.65% and 12.67%. Thus even ideal transform elimination yields only  $1.145\times$  on this workload, whereas ideal Keccak elimination yields  $2.911\times$ . The unchanged permutation count across arms rules out a change in the number of permutations as the explanation for the Keccak delta. It does not establish that every data-dependent path is invariant.

Subtracting both deltas from the baseline leaves 7,262,496 cycles, or 21.68%. Figure 1 shows this *algebraic residual*. It is not a separately instrumented exclusive category or a measured joint ablation. It can contain sampling, polynomial operations outside NTT/INTT, packing, decoding, absorb/squeeze work, allocation, and control. The archive does not split these contributors further, so we do not label all residual time as memory or claim a measured combined Keccak-plus-NTT bound.

Input consistency matters because rejection sampling can change the permutation count. The archived profile now uses KAT coins; earlier 156-permutation measurements and stale narrative percentages are excluded from this attribution. The main SG25 integration instead uses fixed bytes  $0, \dots, 95$  and the library’s serial FIPS-202 path, with 143 permutations in every compared backend. We do not transfer the historical 65.65% share or its bound to that new software/configuration.

### B. ML-DSA: measured key generation and estimated round trips

Table III gives the corresponding ML-DSA results under two library memory budgets. Key generation admits the same

TABLE I

EXPERIMENTAL FAMILIES. ALL USE ONE CORE AND RV32 EXCEPT THE MATCHED RV32/RV64 PORTABILITY ROWS.  $W$  IS RESIDENT WARP CAPACITY,  $T$  IS HARDWARE THREADS PER WARP. A REQUEST COUNT IS A SEPARATE QUANTITY.

| Family                   | $W \times T$  | Workload / software                                                                         | Timing denominator                                  | Scope of valid comparison                            |
|--------------------------|---------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| Historical attribution   | $4 \times 4$  | Single-lane ML-KEM KAT; ML-DSA fixed input                                                  | Instrumented profile's own cycles                   | Component ablation within that profile               |
| C / assembly A/B         | $4 \times 4$  | Same ML-KEM request; reference C or PQRV                                                    | Whole-launch device cycles                          | Optimized software versus C                          |
| Batch and x4 mapping     | $8 \times 4$  | $M = 1, 2, 4, 8$ ; $L = 1, 2, 4$ where archived                                             | Each sweep's own $M = 1, L = 1$ launch              | Request throughput and cache/mapping interaction     |
| SG25 stages / round      | $8 \times 32$ | 32-wide SIMD/ALU; 1 or 8 states                                                             | Synchronized permutation span                       | Shuffle, stage-use ablation, whole round             |
| Complete SG25 ML-KEM     | $8 \times 32$ | Serial FIPS-202; 143 permutations/request                                                   | Whole-launch cycles at fixed $M$                    | SG1, shuffle, stages, whole round, pointer           |
| Final NTT+Keccak ML-KEM  | $8 \times 32$ | W32 NTT/arithmetic; KAT; 140 permutations/request                                           | Whole-launch cycles at fixed $M$                    | Five Keccak backends with all other arms fixed       |
| W32 software / unrolling | $8 \times 32$ | RV32IM; SG1/PQRV/SG5/SG25; looped/unrolled stages                                           | Completed-permutation span; KEM launch at fixed $M$ | Packed-state throughput and equal-unrolling controls |
| Six-parameter board      | $8 \times 32$ | ML-KEM-512/768/1024 and ML-DSA-44/65/87; matched full software mapping                      | Median request makespan at fixed $M$ and workers    | Joint A/E instruction gains within this board cohort |
| Physical implementation  | $8 \times 32$ | RV32 backend builds; matched integer-only RV32/RV64 NTT/Stage/KROUND builds; isolated units | Matched 250/300 MHz constraints                     | LUT/FF and STA-derived frequency within each family  |

TABLE II

ML-KEM-768 COMPONENT ABLATION. SINGLE LANE,  $4w \times 4t$ , SIMX; EVERY ROW RECORDS 144 PERMUTATIONS.

| Variant          | Cycles     | Removed cycles | Share  | Bound |
|------------------|------------|----------------|--------|-------|
| Profile baseline | 33,492,153 | —              | —      | —     |
| Keccak removed   | 11,505,678 | 21,986,475     | 65.65% | 2.911 |
| NTT/INTT removed | 29,248,971 | 4,243,182      | 12.67% | 1.145 |

Source: pqc/results/ablation\_mlkem.csv.

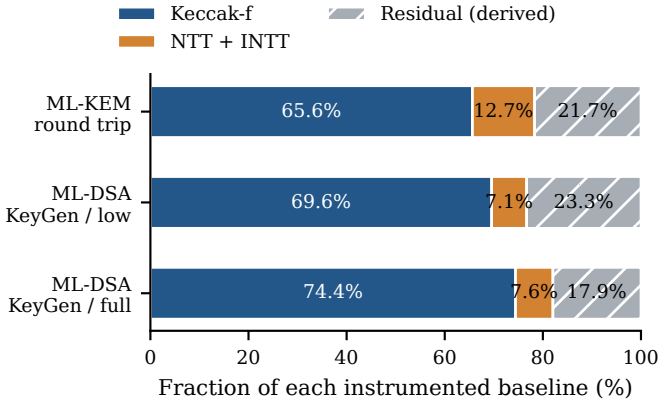


Fig. 1. Historical component attribution from separate ablations. Each bar has its own instrumented denominator. Hatched residuals are obtained by subtraction; additivity has not been measured for these rows.

ablation method: both budgets perform 190 permutations. The Keccak deltas are 29,231,666 cycles for *low* and 29,119,995 for *full*, within 0.4%. The resulting elimination bounds are  $3.285\times$  and  $3.912\times$ . NTT/INTT contribute 7.13% and 7.62%, respectively.

Signing and verification do not admit this substitution.

TABLE III

ML-DSA-65 ATTRIBUTION, SINGLE LANE,  $4w \times 4t$ , SIMX. M: MEASURED COMPONENT ABLATIONS; E: EXTRAPOLATED COMPONENT COSTS. TOTALS ARE MILLIONS OF PROFILE CYCLES.

| Scope / budget    | Total   | Keccak | NTT/INTT | Basis |
|-------------------|---------|--------|----------|-------|
| KeyGen / low      | 42.022  | 69.56% | 7.13%    | M     |
| KeyGen / full     | 39.121  | 74.44% | 7.62%    | M     |
| Round trip / low  | 189.935 | 63.34% | 14.78%   | E     |
| Round trip / full | 142.119 | 62.76% | 18.12%   | E     |

Source: pqc/results/ablation\_mldsa.csv.

Stubbing the hash during signing changes the rejection loop; stubbing it during verification can make `SampleInBall` fail to terminate normally. The round-trip entries therefore multiply call counts by component costs calibrated from key generation. They are estimates, not direct measurements of signing/verification time in each primitive. For the archived input, the low/full round trips execute 782/582 permutations and 103/95 transforms. Their estimated Keccak shares are 63.34%/62.76%; transform shares are 14.78%/18.12%. The memory budget changes the workload composition as well as its total.

These ML-DSA round-trip values describe one input's rejection-loop outcome. An earlier host distribution was not completely archived and had an overflowing histogram; its tail statistics are not used here. A separate, correct pointer-PE experiment reaches  $2.652\times/2.628\times$  round-trip speedups for low/full budgets, supporting the scale of the hash-cost estimate without supplying a missing input distribution.

### C. An optimized software baseline

Reference C alone is insufficient to evaluate an instruction extension. We therefore also use PQRV's hand-written RV32

TABLE IV  
REFERENCE C VERSUS PQRV ASSEMBLY ON THE SAME HISTORICAL  
SINGLE-LANE ML-KEM CONFIGURATION. RATIO IS C/ASSEMBLY.

| Metric               | Reference C | RV32 assembly | Ratio |
|----------------------|-------------|---------------|-------|
| Permutation (cycles) | 151,872.1   | 134,320.2     | 1.131 |
| KEM launch (cycles)  | 33,066,490  | 30,615,811    | 1.080 |
| Retired instructions | 3,467,083   | 3,248,627     | 1.067 |
| Stack peak (bytes)   | 3,944       | 3,776         | 1.044 |

Source: pqc/results/keccak\_baseline\_columns.csv.

TABLE V  
SOFTWARE FORWARD NTT FOR ML-KEM. THE LIBRARY REFERENCE  
TAKES 133,526 CYCLES. MISMATCH COUNT IS ZERO IN ALL ROWS.

| $L$ | Cooperative cycles | Gain vs library | Gain vs coop. $L = 1$ |
|-----|--------------------|-----------------|-----------------------|
| 1   | 211,506            | 0.631           | 1.000                 |
| 2   | 106,297            | 1.256           | 1.990                 |
| 4   | 53,682             | 2.487           | 3.940                 |

Source: pqc/results/ntt\_cooperative.csv.

Keccak implementation, introduced through the library’s native permutation hook [9]. Table IV compares these columns in one historical  $4w \times 4t$  experiment. The NTT implementation is unchanged by the swap.

Assembly improves the permutation by  $1.131\times$  and the complete launch by  $1.080\times$ . Using the profile share in  $1/(1 - f_K + f_K/s_K)$  predicts approximately  $1.082\times$ , a cross-check rather than a replacement for the measured launch. In a separately archived, internally matched three-arm run, reference C, assembly, and the pointer PE take 32,778,558, 30,321,833, and 11,527,027 cycles. That run gives a  $2.844\times$  PE gain over C and  $2.631\times$  over assembly. These counts come from `keccak_ise_simx.csv`; they must not be divided into the totals in Table IV.

#### D. Cooperative NTT as a control

The cooperative forward NTT maps each layer’s 128 butterflies across  $L$  lanes. It uses the library’s arithmetic and twiddle factors and a shared polynomial array, with visibility fences between layers. Every result is checked coefficient by coefficient against the library. Table V reports both the optimized library denominator and the cooperative implementation’s own one-lane denominator.

Four lanes deliver  $3.940\times$  scaling of the cooperative code, but only  $2.487\times$  over the faster single-lane library. Indexing and fences make the cooperative one-lane arm slower. This distinction prevents a  $3.940\times$  scaling result from being presented as the benefit over existing software. The experiment covers a forward transform, not an integrated NTT/INTT extension or complete-KEM speedup.

Microbenchmark setup is another source of bias. The original forward-NTT microbenchmark times coefficient reconstruction with the transform, reporting 160,270.4 cycles rather than the 133,526-cycle transform-only reference. Multiplying such a setup-inclusive cost by application call counts would overstate the transform share. We use application ablation for attribution and transform-only timing for this software control.

TABLE VI  
UNMODIFIED-SOFTWARE REQUEST BATCHING,  $8w \times 4t$ , ONE ACTIVE  
LANE PER REQUEST, L2/L3 OFF. CYCLES ARE MILLIONS PER COMPLETE  
BATCH.

| Scheme | $M$ | Batch cycles | Throughput gain |
|--------|-----|--------------|-----------------|
| ML-KEM | 1   | 33.569       | 1.000           |
| ML-KEM | 2   | 33.575       | 2.000           |
| ML-KEM | 4   | 34.058       | 3.943           |
| ML-KEM | 8   | 40.361       | 6.654           |
| ML-DSA | 1   | 193.767      | 1.000           |
| ML-DSA | 2   | 193.794      | 2.000           |
| ML-DSA | 4   | 196.700      | 3.940           |

Source: pqc/results/batch\_baseline.csv.

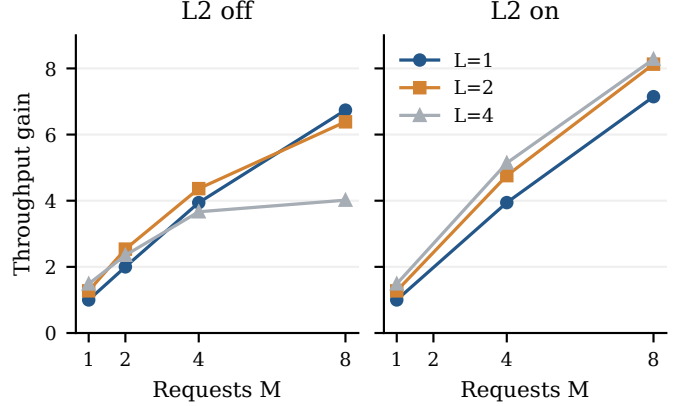


Fig. 2. Historical ML-KEM x4 mapping sweep on  $8w \times 4t$ . Both panels use the sweep’s L2-off,  $M = 1, L = 1$  baseline of 36,614,668 cycles. Source: `mlkem_MxL.csv`. These inputs/backend differ from the KAT ablation and the serial-FIPS-202 SG25 experiment.

#### E. Request batching, lane width, and memory budget

Table VI shows that scheduling independent requests recovers throughput without an instruction extension. ML-KEM reaches  $6.654\times$  at eight requests on an eight-warp, four-thread core. ML-DSA has archived points through four requests in this baseline family. The table reports  $MC(1)/C(M)$ , not reduced latency of one request.

Increasing request concurrency and within-request lane width together can be counterproductive. Figure 2 preserves an independent historical ML-KEM x4 sweep with 156 scalar-equivalent permutations for its one-lane arm. At  $M = 8$ , L2 off, increasing  $L$  from one to four changes the throughput gain from 6.739 to 4.016. With L2 on, the corresponding values are 7.145 and 8.280. This is evidence for a cache-sensitive interaction between mappings, not a universal benefit of wider launches.

ML-DSA adds another software choice. Table VII varies library memory budget and x4 lane width for one request. The full-budget implementation uses an 86,912-byte arena rather than 21,568 bytes. With the low budget, four lanes provide no gain; with the full budget they improve launch time by  $1.601\times$  over that budget’s one-lane arm. Changing both budget and mapping produces  $2.118\times$  over low-budget, one-lane software, without new hardware.

TABLE VII

ML-DSA SOFTWARE MEMORY-BUDGET/WIDTH CONTROL,  $8w \times 4t$ , ONE REQUEST, L2/L3 OFF. GAIN IS RELATIVE TO LOW-BUDGET  $L = 1$ .

| Budget | $L$ | Mcycles | Arena bytes | Gain  |
|--------|-----|---------|-------------|-------|
| low    | 1   | 189.927 | 21,568      | 1.000 |
| low    | 2   | 191.049 | 21,568      | 0.994 |
| low    | 4   | 191.500 | 21,568      | 0.992 |
| full   | 1   | 143.574 | 86,912      | 1.323 |
| full   | 2   | 107.933 | 86,912      | 1.760 |
| full   | 4   | 89.671  | 86,912      | 2.118 |

Source: pqcr/results/mldsa\_ram\_x\_lane.csv.

#### IV. A SUBGROUP KECCAK INTERFACE

##### A. State placement and programming contract

One 32-thread warp holds one state. Lanes 0–24 each hold a 64-bit word as two ordinary RV32 GPR values or one RV64 GPR value. Lanes 25–31 carry no state and return zero. All 32 lanes must execute each collective with the full mask at the same instruction. In particular, guarding the collective with `lane < 25` violates the contract. Guarded loads/stores are allowed if the warp reconverges before the collective.

Each instruction snapshots its complete source vectors and writes one destination vector. It has no pointer operand, memory ordering effect, hidden round counter, or private architectural state. Ordinary RAW hazards, backpressure, and writeback arbitration still apply. Transient pipeline registers are microarchitectural storage and are included in the area cost.

The implementation supports RV32 and RV64 and requires 32 threads, a 32-wide SIMD datapath, and 32 ALU lanes. A narrower packetized ALU would need operand collection across packets; warp width alone is insufficient. The 25-of-32 state layout uses 78.125% of lanes for state words. The remaining lanes still consume execution resources. We do not assume that this utilization loss is free.

##### B. Three stages with single-destination instructions

For  $t = x + 5y$ , let  $A[x, y]$  combine the low/high inputs. All coordinates wrap modulo five. THETA computes

$$C[x] = \bigoplus_{y=0}^4 A[x, y], \quad (4)$$

$$T[x, y] = A[x, y] \oplus C[x - 1] \oplus \text{ROL}_{64}(C[x + 1], 1). \quad (5)$$

`KTHETA.L/H.SG25` each read both state halves and select one output half. Cross-half carry bits from the one-bit rotation are part of the operation; two independent 32-bit reductions would be incorrect.

RHOPI combines a fixed rotation with the lane permutation:

$$B[y, (2x + 3y) \bmod 5] = \text{ROL}_{64}(T[x, y], r[x, y]). \quad (6)$$

At destination  $(x_d, y_d)$ , the source is  $x_s = (x_d + 3y_d) \bmod 5$ ,  $y_s = x_d$ . `KRHOPI.L/H.SG25` select the corresponding result half. Rotation amounts belong to source coordinates, which is essential when implementing the inverse gather.

CHII evaluates each 32-bit half independently:

$$U[x, y] = B[x, y] \oplus (\neg B[x + 1, y] \wedge B[x + 2, y]). \quad (7)$$

Only lane zero then XORs its half of  $RC[q]$ . Unlike THETA/RHOPI, `KCHII.L/H.SG25` read one state half and a uniform round index  $q$ , not both state halves. All lanes, including padding lanes, must supply the same index in  $0, \dots, 23$ .

The six RV32 encodings use `CUSTOM0 (0x0b)`, `funct7=0x06`, with `funct3=0..5`. Each uses two source-vector reads and one destination-vector write. RV64 uses `funct3=0/2/4` for full-word THETA/RHOPI/CHII outputs; THETA and RHOPI leave the second source unused, while CHII reads the uniform round index there. THETA and RHOPI low/high operations must read the same old state:

$$(a_l, a_h) \rightarrow (t_l, t_h) \rightarrow (b_l, b_h) \rightarrow (a'_l, a'_h). \quad (8)$$

Overwriting  $a_l$  before the high THETA operation would break this dependency. Two GPRs describe persistent state capacity, not peak register pressure. The all-stage permutation loop uses five GPRs for state, temporaries, round, and loop limit; its disassembly has no load, store, call, or stack frame.

##### C. ALU integration and the whole-round comparator

Figure 3 contrasts the architectural boundaries. The stage unit sits under `VX_alu_unit` and uses two registered pipeline stages. THETA forms parity then the correction; RHOPI and CHII use the same latency interface. The direct unit latency is two cycles with initiation interval one. These values are not the core’s observed dependent instruction latency.

The source code expresses the fixed cross-lane connections directly. It shares the core’s operand and result interfaces, but the current implementation does *not* establish physical sharing of the existing general shuffle network. A claim of shuffle-network area reuse would require a different integration and its own synthesis evidence.

The independent whole-round extension uses `CUSTOM2 (0x5b)`. On RV32, `funct3=0/1` selects `KROUND.L/H.SG25`; both source vectors contain the old low/high state, and each issue computes one selected half. On RV64, `funct3=0` selects `KROUND.SG25`, one source vector contains the complete state word, and one issue computes the complete round. In both forms, `funct7` carries a literal round index. The four registered stages are column parity, THETA, RHOPI, and CHII/IOTA. Direct latency is four cycles and initiation interval is one. There is no hidden state retained between issued instructions. The low/high pair recomputes from the same old state; it is not a dual-writeback macro-operation.

Literal round encoding expands 24 rounds into 48 RV32 or 24 RV64 instructions. The original Stage kernel retains a round loop with six RV32 or three RV64 collectives, an increment, and a branch per round. Its comparison therefore includes code-generation effects. Table XVI adds a fully expanded Stage control to isolate this asymmetry.

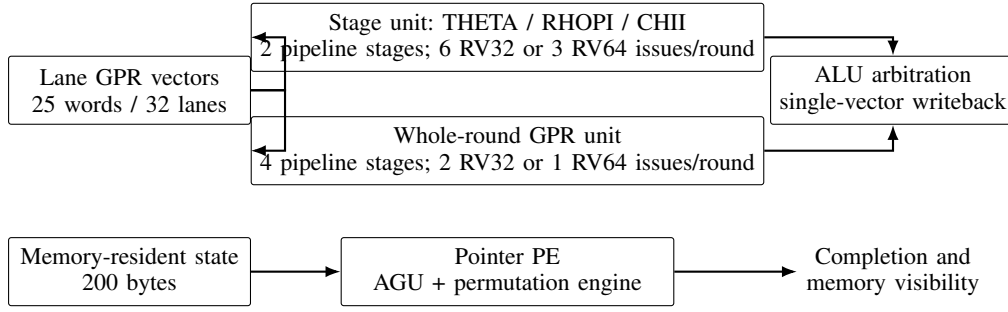


Fig. 3. Alternative Keccak execution boundaries. GPR stages and the whole-round unit use ordinary ALU operand/writeback interfaces. The pointer PE transfers a memory-resident state. Pipeline storage is charged to each implementation.

The pointer KECCAKF baseline remains a full-permutation engine connected through the existing SFU/memory path. It supplies a useful dedicated-engine reference, whereas the GPR variants expose lane-vector data directly to the execution unit. The pointer design requires architectural state transfer through the memory interface for each invocation.

#### D. Full SHAKE and ML-KEM mapping

The subgroup sponge retains state in lane registers across absorb, permutation, and squeeze within a one-shot call. Absorb uses ordinary loads/XORs; squeeze uses ordinary stores. Incremental SHAKE128 saves a word per lane in its context across separate API calls and reloads it on reentry. Thus register residence within a call does not mean the incremental context never touches memory.

For ML-KEM, lane zero executes the surrounding serial library. A TMC boundary and a noinline worker activate the complete warp for FIPS-202, broadcast arguments, run the subgroup sponge, and restore the leader mask on return. A shared allocation arena avoids allocating the same request’s workspace 32 times.

A correctness-first alternative ran the outer KEM redundantly on every lane. We preserve its results as a control. It satisfies output checks but causes excessive memory operations and contention. Removing that redundancy is a software mapping improvement and must not be attributed solely to the Keccak instructions.

### V. EVALUATION

#### A. Correctness and timing methodology

Stage tests cover aliases, zero registers, high/low bit crossings, padding lanes, all round constants, metadata, and backpressure. The mixed stage RTL unit test has 288 RV32 and 144 RV64 transactions; the whole-round test has 192 RV32 and 96 RV64 transactions. Contract tests diagnose partial masks and invalid rounds, plus nonuniform round operands for CHII. These checks diagnose unsupported programming conditions; they do not define a production trap ABI.

Full-round checks use 17 states and 10,200 word comparisons. Complete SHAKE128/256 tests cover 204 input/output cases, including empty messages, unaligned addresses, and

rate boundaries. SimX and XRT integration tests pass. ML-KEM checks return codes, shared-secret agreement, and checksums of the public key, secret key, ciphertext, and shared secret.

Permutation timing launches one state per warp and brackets a dependent chain with synchronized cycle reads. CTA barriers keep initial state loads and final stores outside the timed batch; the measured span is  $\max(t_{\text{end}}) - \min(t_{\text{start}})$ . It includes scheduling skew and any accesses performed inside software stages. For the eight-arm stage study, the marginal cost is

$$c_{\text{marginal}} = \frac{\text{span}(8) - \text{span}(4)}{4M}. \quad (9)$$

The whole-round comparison uses a separate 64-permutation chain and reports  $\text{span}(64)/(64M)$ . These estimators are kept separate in the tables.

Across the 32 matched points in the stage dataset, retired instruction counts agree between SimX and processor RTL; maximum whole-launch cycle difference is 3.24%. Passing this set establishes agreement on the tested cases, not a complete proof of the timing model. The complete ML-KEM XRT checks are integration/correctness results; the internally matched SimX rows supply the application comparisons.

#### B. Eight-way instruction-use ablation

Table VIII uses one hardware build with all six stage encodings enabled. Each arm replaces selected shuffle-software stages with custom instructions. Therefore it measures instruction use, not the physical area obtained by synthesizing only one selected stage.

The complete stage design reduces dependent marginal cost from 19,285.75 to 2,050 cycles, a  $9.41\times$  gain. Eight-warp cost falls from 6,215.59 to 481.50 cycles per completed permutation, a  $12.91\times$  gain. Figure 4 shows that stage benefits depend on the load point: RHOPI gives the largest single-stage latency improvement, while THETA gives the largest single-stage eight-warp improvement. Combined benefits are not obtained by adding the individual speedups.

#### C. Whole-round fusion

Table IX gives the longer-chain comparison. The whole-round unit improves RTL single-state cost from 2,052.219 to 499.266 cycles and eight-warp cost from 481.664 to 117.693

TABLE VIII

RTL MARGINAL PERMUTATION COST FROM EQ. 9. T/R/C INDICATE THETA, RHOPI, CHII INSTRUCTIONS. GAINS USE THE SHUFFLE-ONLY -- ARM AT THE SAME BATCH SIZE.

| Arm | 1-state cyc. | Gain  | 8-warp cyc./perm. | Gain   |
|-----|--------------|-------|-------------------|--------|
| --  | 19,285.75    | 1.000 | 6,215.59          | 1.000  |
| T-  | 12,074.00    | 1.597 | 3,161.56          | 1.966  |
| -R- | 11,173.25    | 1.726 | 3,626.09          | 1.714  |
| -C  | 14,760.50    | 1.307 | 4,650.69          | 1.336  |
| TR- | 5,275.00     | 3.656 | 1,182.75          | 5.255  |
| T-C | 8,818.00     | 2.187 | 2,117.38          | 2.936  |
| -RC | 7,708.00     | 2.502 | 1,953.38          | 3.182  |
| TRC | 2,050.00     | 9.408 | 481.50            | 12.909 |

Source: pqc/results/keccak\_sg25\_stages.csv.

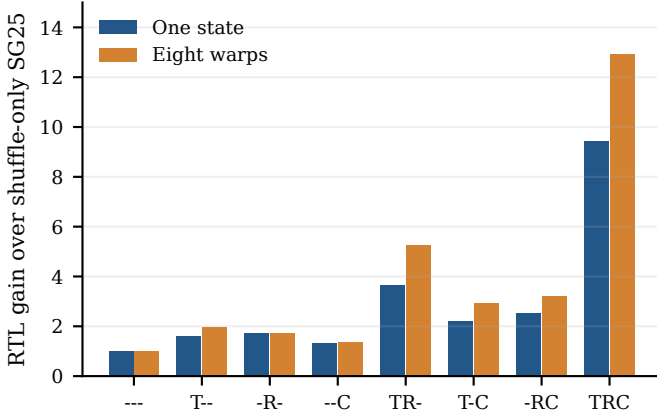


Fig. 4. Stage instruction-use gains on the same  $8w \times 32t$  hardware. The latency and throughput denominators differ as defined in Table VIII.

cycles per completed permutation. This is  $4.110\times$  latency improvement and  $4.093\times$  throughput improvement. This original comparison uses looped Stage code and expanded whole-round code. Table XVI separately controls that difference.

The result makes whole-round fusion a strong implemented comparator. The reduction from six to two issues is not, by itself, a prediction of these cycle ratios. Loop control, register dependencies, issue arbitration, and pipeline timing contribute to the measured sequence.

#### D. Complete ML-KEM

Table X compares complete launches on the same core geometry, cache configuration, fixed coins, and serial FIPS-202 path. All arms perform 143 permutations per request. SG1 and shuffle controls run with the stage extension present but unused. The whole-round arm enables its independent extension; the pointer arm enables PQC and KSG25. These enable sets are recorded in the source data.

The shuffle-only cooperative mapping already improves launch cycles by  $2.422\times$  at one request and  $2.000\times$  at eight requests. Replacing its permutation stages with instructions adds  $1.219\times$  and  $1.383\times$ , yielding total gains of  $2.952\times$  and  $2.766\times$  over SG1. This matched shuffle denominator isolates the instruction contribution from the substantial benefit of mapping state across registers.

TABLE IX

64-PERMUTATION GPR CHAINS ON  $8w \times 32t$ . COSTS ARE  $\text{SPAN}/(64M)$ , EXCLUDING INITIAL/FINAL STATE I/O.

| Model  | Unit        | 1-state cycles | 8-warp cyc./perm. |
|--------|-------------|----------------|-------------------|
| simx   | Stages      | 2,079.312      | 479.463           |
| simx   | Whole round | 502.406        | 121.727           |
| rtlsim | Stages      | 2,052.219      | 481.664           |
| rtlsim | Whole round | 499.266        | 117.693           |

Source: pqc/results/keccak\_kround25.csv.

TABLE X

COMPLETE ML-KEM-768 ON SIMX,  $8w \times 32t$ . CYCLES ARE MILLIONS PER LAUNCH; GAINS USE SG1 AT THE SAME  $M$ .

| Backend             | $M = 1$ | Gain  | $M = 8$ | Gain  |
|---------------------|---------|-------|---------|-------|
| SG1 C               | 34.689  | 1.000 | 53.176  | 1.000 |
| SG25 shuffle        | 14.321  | 2.422 | 26.588  | 2.000 |
| SG25 stages         | 11.749  | 2.952 | 19.228  | 2.766 |
| SG25 whole round    | 11.532  | 3.008 | 18.989  | 2.800 |
| Pointer PE          | 12.430  | 2.791 | 17.014  | 3.126 |
| Redundant outer KEM | 28.684  | 1.209 | 160.039 | 0.332 |

Source: pqc/results/keccak\_sg25\_mlkem.csv.

Whole-round fusion adds only  $1.019\times$  and  $1.013\times$  over the stage backend, despite the roughly fourfold isolated permutation improvement. The single-request instruction count falls from 1,093,915 to 1,073,037, just 1.91%. The evidence identifies a limit to further permutation optimization. The direct final-configuration profile below partitions the named intervals without treating its residual as a single cryptographic primitive.

The stage backend is  $1.058\times$  faster than the pointer PE for one request. At eight requests the pointer PE is  $1.130\times$  faster than stages and  $1.116\times$  faster than the whole-round backend. These results are compatible: a dedicated engine can benefit a saturated request stream even when a register-resident mapping has lower latency for a single request.

#### E. Phase costs and memory-operation evidence

Table XI makes the single-request phase totals explicit. The final column is launch cycles minus the sum of three measured call intervals; it is not an attributed cryptographic primitive. For SG1, key generation, encapsulation, decapsulation, and this remainder account for approximately 28.36%, 31.82%, 38.39%, and 1.43% of the launch. For stages they account for 23.74%, 32.60%, 39.56%, and 4.10%. These phase shares are independent of the historical Keccak/NTT component shares.

Table XII reports whole-launch LSU operation counters and the stack watermark for one request. These counts are neither transferred bytes nor off-chip traffic, and they do not isolate register spills. The stage and whole-round arms have identical load/store counts; fusion primarily changes the permutation instruction sequence. The stage permutation loop itself contains no memory instructions, whereas the surrounding sponge and KEM still access memory.

The redundant outer-KEM control incurs 7,924,185 load/store operations, versus 326,504 for the stage leader

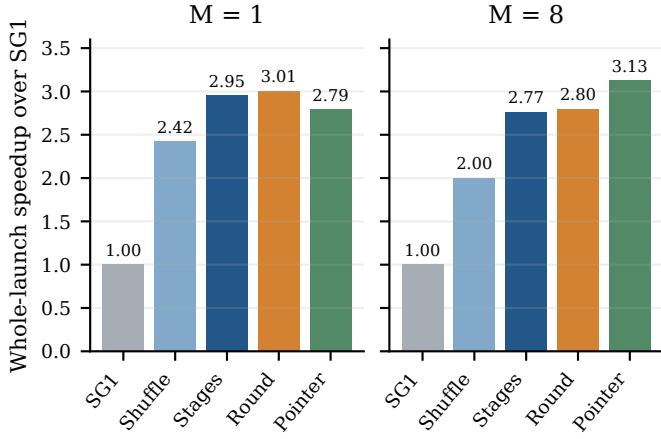


Fig. 5. Complete-launch gains over SG1 at equal request count. The whole-round unit’s large permutation advantage produces a small additional application gain. The pointer PE leads the eight-request launch.

TABLE XI  
SINGLE-REQUEST ML-KEM PHASE INTERVALS, MILLIONS OF CYCLES.  
REMAINDER = WHOLE LAUNCH MINUS PHASE SUM.

| Backend          | KeyGen | Encaps | Decaps | Remainder |
|------------------|--------|--------|--------|-----------|
| SG1 C            | 9.838  | 11.038 | 13.318 | 0.495     |
| SG25 shuffle     | 3.580  | 4.639  | 5.622  | 0.480     |
| SG25 stages      | 2.789  | 3.830  | 4.649  | 0.482     |
| SG25 whole round | 2.722  | 3.762  | 4.567  | 0.482     |
| Pointer PE       | 2.981  | 4.027  | 4.899  | 0.523     |

mapping, approximately 24.3 times as many. At eight requests the redundant version takes 160.039 million cycles, slower than serial SG1. Correct output and a low instruction count alone therefore do not establish an efficient SIMT mapping.

#### F. Final NTT–Keccak integration

Table XIII repeats the six-mapping comparison after fixing the final W32 NTT and cooperative arithmetic configuration. Every arm uses the same combined hardware build, serial FIPS-202 call graph, and FIPS 203 KAT coins. Each request executes 140 Keccak-f permutations, 15 forward NTTs, and nine inverse NTTs; all public keys, secret keys, ciphertexts, and shared secrets match byte-for-byte. Consequently, only the Keccak backend varies.

Stages achieve  $5.258\times/3.646\times$  over SG1 at  $M = 1/8$ . Relative to the matched shuffle implementation, the instruction-only gains are  $1.462\times/1.552\times$ . Whole-round fusion adds  $1.040\times/1.020\times$  over stages. The latency–throughput crossover remains: stages and the whole-round unit beat the pointer PE at  $M = 1$  by  $1.087\times$  and  $1.130\times$ , whereas the pointer PE leads them at  $M = 8$  by  $1.401\times$  and  $1.374\times$ .

All six XRT mappings reproduce the full KAT and SimX retired-instruction counts. The largest whole-launch cycle gap across the matched SimX/XRT controls is 2.71%; the earlier Stage and whole-round M1 gaps were 0.004% and 0.093%. These runs cover the AFU and processor RTL. Table XIII uses a common all-enabled core for the func-

TABLE XII  
SINGLE-REQUEST WHOLE-LAUNCH MEMORY-OPERATION COUNTS AND STACK WATERMARKS. ALL BACKENDS USE A 19,232-BYTE PEAK ALLOCATION ARENA.

| Backend             | Loads     | Stores    | Stack bytes |
|---------------------|-----------|-----------|-------------|
| SG1 C               | 628,039   | 512,165   | 1,912       |
| SG25 shuffle        | 521,343   | 189,538   | 2,184       |
| SG25 stages         | 164,422   | 162,082   | 2,156       |
| SG25 whole round    | 164,422   | 162,082   | 2,156       |
| Pointer PE          | 152,797   | 136,038   | 1,420       |
| Redundant outer KEM | 3,994,787 | 3,929,398 | 1,868       |

TABLE XIII  
FINAL INTEGRATED ML-KEM-768 ON XRT,  $8w \times 32t$ . CYCLES ARE MILLIONS PER LAUNCH; GAINS USE SG1 AT THE SAME  $M$ .

| Backend          | $M = 1$ | Gain  | $M = 8$ | Gain  |
|------------------|---------|-------|---------|-------|
| SG1 C            | 28.446  | 1.000 | 45.674  | 1.000 |
| PQRV             | 24.672  | 1.153 | 37.753  | 1.210 |
| SG25 shuffle     | 7.909   | 3.597 | 19.440  | 2.350 |
| SG25 stages      | 5.410   | 5.258 | 12.527  | 3.646 |
| SG25 whole round | 5.203   | 5.467 | 12.280  | 3.719 |
| Pointer PE       | 5.878   | 4.840 | 8.939   | 5.109 |

Source: pqc/results/keccak\_ntt\_unified\_xrt.csv.

tional and cycle comparison; the historical SimX rows remain in keccak\_ntt\_unified\_mlkem.csv. Section VI uses four separate builds that retain the same NTT and enable either no Keccak hardware, stages, whole round, or the pointer PE.

#### G. Six-parameter 200 MHz board sweep

Table XIV measures full ML-KEM key generation, encapsulation, and decapsulation or ML-DSA key generation, signing, and verification. All six sets use one RV32,  $8w \times 32t$  board image and the same complete software mapping in each arm. A executes software Keccak and NTT; E adds Stage, NTT, and arithmetic instructions. The KEM mapping includes warp-level sampling, encoding, noise, and linear operations; the DSA mapping includes warp-level sampling, SHAKE extraction, and signing arithmetic. Each cell has one warmup and five timed runs, with byte-exact KAT checks. The table divides median A makespan by median E makespan at the same request count and worker count. All DSA inputs in this sweep start at index 1.

The joint gain spans  $1.910\text{--}2.365\times$  at one request and  $1.914\text{--}2.126\times$  at eight. Both ML-DSA-44 and ML-DSA-65 remain below  $2\times$  in these matched comparisons; the four other sets exceed  $2\times$  at both request counts. These ratios include all three instruction families and must not be attributed to Keccak alone. The earlier incomplete-mapping parameter sweep uses different software binaries; its A/E ratios are not pooled with this cohort. The historical DSA-65 board ablation also starts from a different signing input.

#### H. Matched W32 software and unrolling controls

The additional controls hold RV32IM, disabled F/D, W8T32, final NTT, and the Stage/KROUND-enabled core fixed. Table XV reruns reference C, the vendored PQRV

TABLE XIV

MATCHED FULL-MAPPING A/E ROUND-TRIP SPEEDUPS ON THE 200 MHZ V80 BOARD.  $M = 1$  USES ONE WORKER;  $M = 8$  USES EIGHT WORKERS.

| Parameter set | $M = 1$ | $M = 8$ |
|---------------|---------|---------|
| ML-KEM-512    | 2.341   | 2.126   |
| ML-KEM-768    | 2.314   | 2.004   |
| ML-KEM-1024   | 2.365   | 2.013   |
| ML-DSA-44     | 1.966   | 1.966   |
| ML-DSA-65     | 1.910   | 1.914   |
| ML-DSA-87     | 2.150   | 2.023   |

Source:

pqc/results/v80\_hw\_validation/parameter\_full\_board\_200mhz/pqc\_parameters\_runs.csv.

TABLE XV

MATCHED W32 SOFTWARE CYCLES PER COMPLETED PERMUTATION (SIMX).  $S$  IS PACKED STATES/WARP; W1/1 AND W8/1 USE ONE STATE/WARP. SG5 TIMING HAS THE MODEL DISCREPANCY DISCUSSED BELOW.

| Mapping | $S$ | W1/1      | W8/1     | W8/ $S$  |
|---------|-----|-----------|----------|----------|
| SG1 C   | 32  | 156,737.0 | 35,637.8 | 66,240.2 |
| PQRV    | 32  | 134,438.0 | 24,874.3 | 54,992.3 |
| SG5     | 6   | 57,415.0  | 14,307.6 | 11,924.1 |
| SG25    | 1   | 19,369.1  | 6,801.2  | 6,801.2  |

Source: pqc/results/keccak\_w32\_controls.csv.

RV32IM assembly, the existing SG5 column mapping, and SG25 shuffles. SG1/PQRV can pack 32 independent states in a warp, SG5 six, and SG25 one. SG5 uses its existing memory transpose and two fences per round; this is a specific software mapping, not a bound on all five-lane implementations. The two occupancies are capacity controls, not an exhaustive occupancy search. CTA barriers exclude every participating warp’s initialization and output traffic from the measured span. The host checks all words after eight dependent permutations per state.

A fixed-input XRT control runs 64 completed permutations with one state per warp. SG1, PQRV, the barrier-based SG5, shuffle SG25, unrolled Stage, and KROUND all pass output and guard checks on one all-enabled core. Their respective cycles per permutation are 37,154.1, 24,782.0, 15,910.5, 6,753.4, 389.8, and 119.9. This controls input and geometry, not each mapping’s maximum packed throughput. The raw spans and hashes are in keccak\_w32\_matched\_xrt.csv.

Table XVI removes the loop-unrolling asymmetry: the Stage control expands all 24 rounds, as required by KROUND’s literal-round encoding. Disassembly shows 144 Stage instructions or 48 whole-round instructions per permutation, with no round-loop branch in either expanded function. Whole round retains a  $3.155\times$  single-warp speedup and  $3.300\times$  higher eight-warp throughput. The original looped comparison therefore includes a software control-flow effect, but that effect does not explain the entire fusion gain.

The same PQRV hook and expanded Stage function enter the final ML-KEM KAT path in Table XVII. All arms preserve 140 scalar permutations, 15 NTTs, and nine inverse NTTs per request. PQRV improves the C launch by  $1.122\times/1.170\times$  at  $M = 1/8$ ; expanded stages still improve PQRV by

TABLE XVI

EQUAL-UNROLLING CONTROL, 64 DEPENDENT PERMUTATIONS PER STATE (SIMX). ONE STATE/WARP; CYCLES ARE PER COMPLETED PERMUTATION.

| Backend          | ISE issues/perm. | W1 cycles | W8 cycles |
|------------------|------------------|-----------|-----------|
| Stage / loop     | 144              | 2,079.3   | 479.5     |
| Stage / unrolled | 144              | 1,584.9   | 401.7     |
| Whole round      | 48               | 502.4     | 121.7     |

Source: pqc/results/keccak\_w32\_controls.csv.

TABLE XVII

MATCHED RV32IM ML-KEM CONTROLS (SIMX). MILLIONS OF WHOLE-LAUNCH CYCLES; GAINS USE PQRV AT THE SAME REQUEST COUNT.

| Backend          | $M = 1$ | Gain  | $M = 8$ | Gain  |
|------------------|---------|-------|---------|-------|
| SG1 C            | 27.739  | 0.891 | 44.212  | 0.855 |
| PQRV             | 24.714  | 1.000 | 37.786  | 1.000 |
| SG25 shuffle     | 7.900   | 3.128 | 19.204  | 1.968 |
| Stage / loop     | 5.410   | 4.569 | 12.170  | 3.105 |
| Stage / unrolled | 5.342   | 4.626 | 12.336  | 3.063 |
| Whole round      | 5.198   | 4.754 | 11.938  | 3.165 |

Source: pqc/results/mlkem\_w32\_controls.csv.

$4.626\times/3.063\times$ . Stage unrolling reduces M1 cycles by 1.25% but increases M8 cycles by 1.36%. Whole round improves the expanded Stage launch by only  $1.028\times/1.033\times$ . Fewer permutation instructions thus do not imply a proportional application gain. Selected XRT controls reproduce reference outputs and KATs with the same ELF and exact retired instruction counts. Excluding SG5, their maximum whole-launch gap from SimX is 3.37%.

The two SG5 XRT checks differ by 6.93% and 12.22%, exceeding the default 5% timing-model gate despite correct outputs. The implementation retains cache-flushing fences, whereas SimX releases a fence after pending LSU requests drain without modeling the full RTL cache flush. SG5’s SimX timing therefore remains a prediction with this limitation; it is not reported as a model-parity pass. No RTL or physical design changes are involved in these controls.

A synchronization experiment replaced both transpose fences with warp-private BARs, which drain the shared core LSU without flushing the cache. The original gaps fell to 1.39% and 1.19%, but expanded eight-warp tests still differed by 5.44% for six states/warp with two permutations and 9.63% for one state/warp with eight permutations. Across 31 pairs including CTA barriers, LMEM, and a single-barrier control, all outputs and retired instructions agreed, but only 20 pairs passed timing. These historical results and diagnostic patches remain in keccak\_sg5\_sync.csv and its source archive.

Trace comparison then identified a separate cache-model error: RTL forwards a staged fill before array installation, whereas SimX waited for the array pipeline. Capturing and forwarding the fill at admission corrects this ordering without changing RTL or timing constants. A cache regression checks array-depth independence, read/store ordering, response back-pressure, and write-through stores around fill arrival. With the same SG5 BAR binary, all 11 repeated SimX/XRT

TABLE XVIII

DIRECT XRT M1 PHASE SHARES IN THE FINAL RV32IM CONFIGURATION. POLY. = MULCACHE + BASEMUL + REDUCE; RESIDUAL INCLUDES PROBE OVERHEAD.

| Backend     | Perm. | Absorb | Squeeze | NTTs  | Poly. | Residual |
|-------------|-------|--------|---------|-------|-------|----------|
| Stages      | 4.96% | 7.42%  | 4.24%   | 2.34% | 4.53% | 76.50%   |
| Whole round | 1.27% | 7.65%  | 5.26%   | 2.40% | 4.67% | 78.76%   |
| Pointer PE  | 0.52% | 9.52%  | 17.26%  | 2.11% | 4.18% | 66.40%   |

Source: pqc/results/mlkem\_phase\_profile.csv.

pairs now agree on instructions and satisfy the unchanged 5% whole-launch gate; the two eight-warp gaps become 4.07% and 3.29%, with a maximum of 4.23%. One isolated permutation interval still differs by 5.63%. The new CSV `keccak_sg5_cache_forward.csv` records this follow-up separately. Full fence-flush modeling remains incomplete; the default fence implementation and historical performance tables, including Table XV, are unchanged.

#### I. Direct final-configuration phase profile

Table XVIII measures mutually exclusive M1 intervals in the final RV32IM W8T32 configuration. Absorb and squeeze exclude nested permutation cycles; NTT/INTT and the listed cooperative polynomial operations are timed directly. The residual is the exact difference from the instrumented request interval, so it includes sampling, encoding, comparison, control, memory, wrappers, allocation, and interval-probe overhead. Each row uses the corresponding Stage-only, KROUND-only, or pointer-only extension build and passes the complete KAT.

The pointer PE reduces permutation to 0.52% of its instrumented interval, while absorb plus squeeze account for 26.78%. The corresponding sponge shares are 11.66% for stages and 12.90% for whole round. This explains why replacing stages with a whole round has little application effect and why pointer permutation latency alone does not predict the saturated result. SimX/RTL and XRT/RTL interval gaps remain below 0.78% and 0.08%, respectively, with exact instruction agreement. Probe overhead is 7.189%, 8.299%, and 6.632% for the three rows, so headline speedups continue to use the uninstrumented runs.

#### J. Matched integer-only RV32/RV64 portability

Table XIX recompiles Stage and KROUND at RV32IM and RV64IM while holding the source revision, W8T32 geometry, KAT input, cache configuration, NTT, and cooperative arithmetic fixed. F and D are disabled at both XLENs; ELF attributes and disassembly confirm soft-float IM binaries without floating-point or atomic instructions. SimX, RTL, and XRT reproduce every M1 KAT with exact retired-instruction agreement. All eight SimX and subsequent Stage/KROUND XRT requests also pass byte-for-byte.

RV64 reduces Stage instructions by 7.00% and KROUND instructions by 5.73%. Their M1 cycles fall by 4.07% and 2.98%, respectively, while M8 cycles rise by 0.46% and

TABLE XIX

MATCHED INTEGER-ONLY ML-KEM AT RV32 AND RV64 ON SIMX. CYCLES ARE MILLIONS PER COMPLETE LAUNCH.

| ISA           | M1 instr. | M1 cycles | M8 instr. | M8 cycles |
|---------------|-----------|-----------|-----------|-----------|
| Stage RV32IM  | 436,262   | 5.410     | 3,490,096 | 12.170    |
| Stage RV64IM  | 405,716   | 5.190     | 3,245,728 | 12.226    |
| KROUND RV32IM | 415,822   | 5.198     | 3,326,576 | 11.938    |
| KROUND RV64IM | 391,996   | 5.044     | 3,135,968 | 11.971    |

Source: pqc/results/rv32im\_rv64im\_keccak.csv.

0.27%. The saved Keccak issues are only part of the complete launch: NTT, sampling, encoding, memory traffic, and remaining arithmetic do not halve with the issue count. At M8, the higher cycles per retired instruction offset the smaller stream. The routed cost of the same integer-only NTT/Stage/KROUND configurations appears in Table XXII.

The additional M8 XRT runs retire the same instructions as SimX and complete all eight KATs. Stage takes 12.527/12.476 million cycles at RV32/RV64; KROUND takes 12.280/12.302 million. The RV64-minus-RV32 cycle changes are -0.40% and 0.18%, respectively. Dividing these cycles by each independent post-route STA frequency gives estimated RV64 time increases of 3.40% and 0.66%. These are single-seed estimates, not measured FPGA execution times; RV64 Stage and KROUND do not meet the 250 MHz target. The M8 XRT logs and hashes are indexed in `rv32im_rv64im_keccak_xrt_m8.csv`.

## VI. IMPLEMENTATION COST

#### A. Final NTT architecture and selection

The final NTT maps one transform to one 32-lane warp. Coefficients remain in lane GPRs across the seven layers. The three lane-local layers use `NTTMUL.K`; the four cross-lane layers use `CT/GS NTTBF` operations at XOR distances 16, 8, 4, and 2. A half-width bank shares 16 signed  $16 \times 16$  multipliers across the warp. CT retains latency/II 6/1, while GS and W32 `NTTMUL.K` use 7/2. Static routing implements the five legal encoded XOR distances; this kernel uses four of them.

Table XX records the matched architecture sweep that selected this design. Every row in that historical sweep includes the same pointer Keccak PE, so the first row is the no-NTT-ISE denominator for this table only. The registered full bank closes 250 MHz. Sharing the multipliers reduces its full-core LUT count by 4.14% while changing matched complete-KEM cycles by at most 0.041%. Relative to the historical no-NTT-ISE core, the selected half bank adds 2.02% LUTs, 2.14% FFs, and 16 DSPs without adding BRAM.

The full-bank-to-half-bank comparison holds instruction semantics and the kernel binary fixed, but the table predates the independently gated merged source used below. We therefore use it to justify the NTT architecture selection and do not subtract its absolute counts from the new Keccak rows.

TABLE XX

HISTORICAL MATCHED V80 NTT ARCHITECTURE SWEEP AT 250 MHZ. DELTAS USE THE NO-NTT-ISE ROW; ALL ROWS INCLUDE THE SAME POINTER KECCAK PE. WNS IS NS.

| NTT hardware | LUT     | $\Delta$ LUT | FF      | $\Delta$ FF | DSP | WNS   |
|--------------|---------|--------------|---------|-------------|-----|-------|
| No NTT ISE   | 334,917 | –            | 265,187 | –           | 160 | 0.000 |
| Full bank    | 356,447 | +21,530      | 272,430 | +7,243      | 192 | 0.000 |
| Half bank    | 341,680 | +6,763       | 270,865 | +5,678      | 176 | 0.000 |

Source: pqc/results/ntt\_v80\_ppa.csv.

TABLE XXI

MATCHED FULL-CORE V80 POST-ROUTE RESULTS AT 250 MHZ. DELTAS USE NTT ONLY AS THE DENOMINATOR. WNS IS NS.

| Core              | LUT     | $\Delta$ LUT | FF      | $\Delta$ FF | DSP | WNS   |
|-------------------|---------|--------------|---------|-------------|-----|-------|
| NTT only          | 330,530 | –            | 267,838 | –           | 176 | 0.000 |
| NTT + stages      | 334,561 | +4,031       | 269,806 | +1,968      | 176 | 0.006 |
| NTT + whole round | 332,710 | +2,180       | 274,237 | +6,399      | 176 | 0.003 |
| NTT + pointer PE  | 341,751 | +11,221      | 271,058 | +3,220      | 176 | 0.004 |

Source: pqc/results/ntt\_keccak\_backend\_ppa.csv.

### B. Matched full-core Keccak backends

We target `xcv80-lsva4737-2MHP-e-S` with Vivado 2025.1, using post-route physical optimization. Table XXI compares four independent full-core builds under the same 250 MHz target. Each contains the selected NTT and arithmetic path. The NTT-only row is the common physical denominator; each other row enables exactly one Keccak hardware backend. The stage overhead is 4,031 LUT (1.22%) and 1,968 FF (0.73%). Whole round adds 2,180 LUT (0.66%) and 6,399 FF (2.39%), whereas the pointer PE adds 11,221 LUT (3.39%) and 3,220 FF (1.20%). BRAM and DSP counts are unchanged.

These independently gated builds supply the area denominator for the final ML-KEM comparison. They do not replace the common all-enabled binary used to measure cycles, whose purpose is to keep software and timing-model context identical. Post-route full-core differences include integration, placement, and routing effects in addition to the selected unit hierarchy. All four builds close 250 MHz, with WNS from 0.000 to 0.006 ns. The routed stage, whole-round, and pointer hierarchies contain 902/2,025, 2,503/6,130, and 10,414/4,556 LUT/FF, respectively. The whole-round critical path remains in issue-to-integer-ALU routing rather than inside the new unit.

### C. Matched integer-only RV32/RV64 implementation

Table XXII holds the V80 device, W8T32 geometry, 250 MHz target, Vivado version, optimization level, and selected half-bank NTT fixed. F and D are disabled at both XLENs. It compares NTT-only, Stage, and KROUND builds; every row is an independent, non-incremental post-route run with zero routing errors. Negative WNS rows are reported as timing misses rather than closed implementations. RV64 Stage has WNS -0.148 ns at 241.1 MHz, while RV64 KROUND reaches -0.013 ns and 249.2 MHz.

Relative to RV32 NTT-only, the RV64 NTT-only full-core resource changes are 94.19% for LUTs, 73.19% for FFs, 96.62% for BRAM tiles, and -85.71% for DSPs. Both FPU hi-

TABLE XXII

MATCHED INTEGER-ONLY RV32/RV64 V80 POST-ROUTE RESULTS. BRAM IS TILE-EQUIVALENT, WNS IS NS, AND STA FREQUENCY IS MHZ UNDER A 250 MHZ TARGET.

| Core                   | LUT     | FF      | BRAM  | DSP | WNS    | STA MHz |
|------------------------|---------|---------|-------|-----|--------|---------|
| RV32 NTT only          | 220,950 | 131,798 | 133.0 | 112 | 0.004  | 250.3   |
| RV32 NTT + stages      | 223,444 | 133,512 | 133.0 | 112 | 0.004  | 250.3   |
| RV32 NTT + whole round | 222,935 | 138,065 | 133.0 | 112 | 0.007  | 250.4   |
| RV64 NTT only          | 429,067 | 228,266 | 261.5 | 16  | 0.000  | 250.0   |
| RV64 NTT + stages      | 432,843 | 232,350 | 261.5 | 16  | -0.148 | 241.1   |
| RV64 NTT + whole round | 431,004 | 234,240 | 261.5 | 16  | -0.013 | 249.2   |

Source: pqc/results/rv32im\_rv64im\_keccak\_ppa.csv.

TABLE XXIII

ISOLATED UNITS ON V80, 300 MHZ TARGET.

| Unit        | LUT   | FF    | STA $f_{\max}$ (MHz) |
|-------------|-------|-------|----------------------|
| Pointer PE  | 3,223 | 1,607 | 368.4                |
| Stages      | 2,821 | 2,101 | 323.7                |
| Whole round | 2,610 | 6,282 | 359.6                |

erarchies are absent, while the narrow NTT remains at 16 DSP. The DSP decrease comes from the general multiply/divide unit: the RV32 implementation uses 96 DSPs there, whereas the serial RV64 implementation uses 0.

Within RV32, Stage adds 2,494 LUT (1.13%) and 1,714 FF (1.30%); KROUND adds 1,985 LUT (0.90%) and 6,267 FF (4.76%). Within RV64, the corresponding increments are 3,776/4,084 LUT/FF for Stage and 1,937/5,974 LUT/FF for KROUND. None changes BRAM or DSP count.

The wide core itself must also be charged. The historical 300 MHz base configuration grid uses 42,553 LUTs at  $8w \times 4t$ , 148,827 at  $8w \times 16t$ , and 325,407 at  $8w \times 32t$ . Consequently, a small extension overhead relative to the chosen wide core is not the cost of widening a narrow GPU. The SG25 study fixes the wide core to isolate instruction granularity.

### D. Isolated-unit FPGA and ASIC comparisons

Tables XXIII and XXIV compare isolated units. The pointer unit performs a complete permutation, stages execute one selected RV32 stage instruction, and the whole-round unit executes one selected half of a round. Unit area is therefore not a same-operation latency comparison.

The whole-round unit uses 7.48% fewer isolated FPGA LUTs than stages, but 2.990 times the FFs. On the predictive ASAP7 library [10], it uses 2.123 times the total cell area and 2.989 times the sequential area. Its larger register footprint is visible after full-core FPGA integration, but LUT-only normalization largely hides it.

At common 250 MHz, the original 64-chain RTL results with looped Stage code and the corresponding full-core LUT counts give a whole-round versus stage throughput/LUT ratio of

$$\frac{481.664}{117.693} \times \frac{334,561}{332,710} \simeq 4.115. \quad (10)$$

Thus the whole-round unit also wins this FPGA efficiency metric for pure permutation. It would be misleading to claim

TABLE XXIV

ISOLATED ASAP7 RVT, TT, 1.00 V, 25°C. YOSYS 0.57 / OPENSTA 2.7.0; MATCHED 500 MHz MAPPING CONSTRAINT. AREA IS  $\mu\text{m}^2$ ; THESE ARE MAPPED, NOT ROUTED, ASIC RESULTS.

| Unit        | Total area | Sequential area | STA MHz |
|-------------|------------|-----------------|---------|
| Pointer PE  | 1,723.254  | 470.642         | 2,187.2 |
| Stages      | 1,832.123  | 612.943         | 1,882.9 |
| Whole round | 3,889.186  | 1,831.831       | 2,041.2 |

that the stage design wins all area-normalized metrics because it has fewer pipeline registers. Conversely, dividing the same pure-permutation gain by isolated ASIC area does not yield a measured full-ASIC application efficiency. We omit power/energy claims because no workload-activity-based power measurement is available.

## VII. RELATED WORK AND INTERPRETATION

PQRV optimizes Keccak and polynomial arithmetic for several RISC-V ISA variants [9]. It motivates an assembly baseline rather than reliance on portable C alone. Bolat et al. study a custom SHA-3 instruction and its pipeline/storage integration in a RISC-V CPU [11]. These approaches show that instruction acceleration and baseline selection already have substantial prior work; the extra question here is how cooperating SIMT lane registers change the design boundary.

ML-Cube combines register-oriented polynomial computation with GPU execution and describes a 25-thread shuffle mapping for Keccak in Sec. 5.1.1 [5]. We use this mapping as prior art and as a software denominator. We do not compare its commercial-GPU timing numerically against a single FPGA soft core.

Listing 3 of NVIDIA’s published application describes `keccak1.sync.aligned.u64`, where lanes 0–24 hold 64-bit input/output words and a warp-collective invocation performs one round [6]. The text also discusses multiple-round forms. It supplies a concrete semantic predecessor, not measurements for our platform. Our comparator has explicitly implemented RV32 low/high and RV64 full-word destinations, literal rounds, a four-stage datapath, and single-vector writeback. Its timing and PPA numbers belong to this implementation and are not measurements of NVIDIA hardware.

The useful contribution is the controlled granularity study and its end-to-end consequences. Stages expose individually testable and replaceable collectives at lower sequential cost. Whole-round fusion provides the best measured GPR permutation throughput and competitive FPGA LUT efficiency. The pointer PE leads the saturated ML-KEM workload despite architectural memory transfer. Which point to adopt depends on the target workload and the area metric, not solely on the number of custom instructions.

## VIII. LIMITATIONS AND NEXT MEASUREMENTS

Historical component fractions apply to their fixed input, instrumented build, and narrow-core configuration. The direct

final-configuration profile separates the named cryptographic intervals, but its 6.6–8.3% probe overhead is part of the residual. It does not split sampling, encoding, comparison, control, memory, wrappers, or allocation. At eight requests, per-request intervals overlap and therefore cannot be summed; only the uninstrumented global makespan is reported.

The matched W32 software controls cover one upstream RV32IM assembly routine and one SG5 column mapping with synchronization controls. They do not establish the best possible software implementation. Packed-state permutation throughput is distinct from complete-request throughput; SG5 has no end-to-end ML-KEM integration here.

The six-parameter board sweep covers complete ML-DSA requests, but it does not isolate an ML-DSA Keccak collective or characterize mixed-input batch distributions. Its fixed-input DSA signing times can depend on rejection sampling. The corresponding full-mapping SimX/XRT parity campaign remains incomplete, so the board results do not establish timing-model parity across all six sets. We report no quantiles from the incomplete host sweep.

Evidence includes XRT integration simulation, a 200 MHz V80 board sweep, and out-of-context full-core FPGA routing. The resident board image was reused; clock readback confirms 200 MHz, but VRT metadata does not independently attest the resident PDI identity. Complete-system PPA, full-core routed ASIC results, and activity-driven energy remain outside the demonstrated scope. Each XLEN physical row is one implementation seed. Register residence alone does not establish side-channel resistance.

## IX. CONCLUSION

The software baseline determines both which primitive to accelerate and which speedup is meaningful. On the historical ML-KEM profile, Keccak accounts for 65.65% and NTT/INTT for 12.67%; independent-request batching and optimized software provide additional, separately measured baselines.

On a fixed RV32  $8w \times 32t$  Vortex core, three subgroup stages convert the 25-word Keccak state into an explicit GPR collective interface. They improve permutation throughput by  $12.91\times$  over shuffles and, with the final NTT path fixed, complete ML-KEM by  $5.258\times/3.646\times$  over SG1 at one/eight requests. The matched full-mapping board sweep extends the joint instruction evaluation to six ML-KEM/ML-DSA parameter sets, with  $1.910\text{--}2.365\times$  at one request and  $1.914\text{--}2.126\times$  at eight. With equal round unrolling, whole-round fusion adds  $3.300\times$  permutation throughput but only  $1.028\times/1.033\times$  application speedup. The pointer PE remains strongest in the earlier eight-request launch. Direct XRT intervals show that pointer permutation is only 0.52% of the instrumented request, whereas its absorb and squeeze paths consume 26.78%. Integer-only RV64 halves the Keccak issue count, reduces complete-launch instructions by 5.7–7.0%. SimX slightly raises both eight-request cycle results; matched XRT instead shows a small Stage cycle reduction and a small KROUND increase. At individually achieved STA frequencies, RV32 still has the shorter estimated time for both. The result is

an implemented set of design choices with measured tradeoffs, rather than a universal ranking inferred from one primitive benchmark.

## APPENDIX A

### DATA PROVENANCE AND REPRODUCTION

All local measurements quoted here are archived under `pqc/results/`. The companion README maps tables to source files. The generator recomputes ratios, checks board medians against five raw repeats, checks ablation differences and phase sums, and emits vector figures and a SHA-256 input manifest. It uses data rows rather than historical narrative percentages. The new W32 controls identify their source commit and executable hashes. Some historical collective datasets were measured with local changes; HEAD and input hashes cannot recover their missing source snapshots. Build with `make` in `pqc/docs/paper`; the portable source package also contains frozen generated tables and figures.

### REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “Module-lattice-based digital signature standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] —, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [4] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [5] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [6] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [7] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>
- [8] —, “mldsa-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mldsa-native>
- [9] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [10] The OpenROAD Project, “ASAP7 predictive process design kit and standard-cell libraries,” Design collateral repository, 2026. [Online]. Available: <https://github.com/The-OpenROAD-Project/asap7>
- [11] A. Bolat, S. Sezer, K. McLaughlin, and H. Hui, “Microarchitecture design and benchmarking of custom SHA-3 instruction for RISC-V,” *arXiv:2508.20653*, 2025.